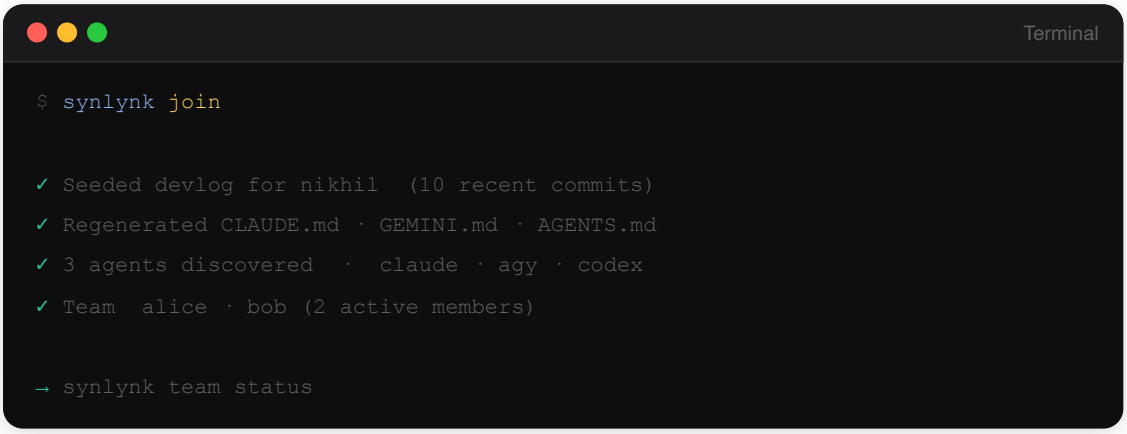
THE OS FOR MULTI-AGENT DEVELOPMENT

synlynk quick start.

From zero to a context-aware, cost-tracked, multi-agent workgroup in under two minutes.

v0.9.2 Team Onboarding + Consensus

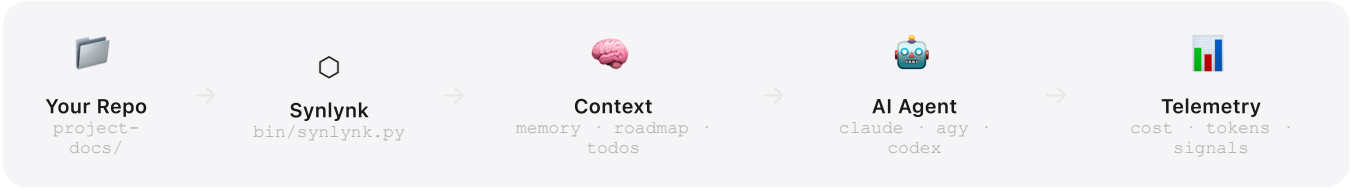


Zero dependencies · Python 3 stdlib only · Claude · AGY · Codex · Team onboarding · Sentinel guard · Multi-agent consensus

WHAT IS SYNLYNK?

Your AI agents, working together.

Synlynk is a single-file Python CLI that wraps AI agents — Claude, Gemini, Codex — and makes them project-aware, cost-tracked, and safe to run in parallel. One file. Zero dependencies. Works anywhere Python 3 runs.



Why Synlynk?

Context-Aware by Default

Every agent invocation automatically receives your roadmap, memory, and todos. No more copy-pasting context into every session.

Zero Surprise Bills

Budget guard blocks execution the moment you hit your daily or session limit. Real-time cost tracking on every exec.

Hallucination Sentinel

Automatically detects flatline loops, success loops, and quota exhaustion. Alerts before damage is done.

Parallel Dispatch

Launch Claude, Gemini, and Codex simultaneously on different tasks. Monitor all jobs from one CLI while your terminal stays free.

Team-Ready

Single or team mode. Per-user devlogs with git attribution. Shared roadmap and memory visible to every agent in the repo.

Zero Dependencies

Pure Python 3 stdlib. No pip install, no Docker, no build step. One file — works in any environment where Python 3 runs.

1	3	0	∞
FILE — NO BUILD STEP	AI AGENTS SUPPORTED	PIP DEPENDENCIES	PARALLEL AGENT JOBS

GETTING STARTED

Up and running in 60 seconds.

1 **Clone & install globally**

Adds `synlynk` to your PATH via `~/.synlynk/bin/`. No sudo, no pip, no build step required.

install

```
$ git clone https://github.com/nikhilsoman/synlynk
$ cd synlynk && ./install.sh

✓ Installed to ~/.synlynk/bin/synlynk
✓ PATH updated in ~/.zshrc
✓ synlynk 0.4.1 ready
```

2 **Bootstrap any project**

Run inside any repo. Creates `project-docs/` and writes `CLAUDE.md`, `GEMINI.md`, `AI_INSTRUCTIONS.md`. Safe to re-run — skips existing files.

synlynk init

```
~/my-project $ synlynk init

Mode: solo | team? solo

✓ project-docs/roadmap.md · memory.md · todo.md · costs.md
✓ CLAUDE.md · GEMINI.md · AGENTS.md · AI_INSTRUCTIONS.md
✓ Tracked 7 instruction files in .synlynk/instructions.json
✓ Agents: claude · agy · codex

→ synlynk instructions status
```

3 **Run your first context-injected session**

Synlynk prepends your project context — the AI already knows your roadmap, decisions, and open tasks before you type a word.

synlynk exec

```
$ synlynk exec claude

📋 Context injected   (memory · roadmap · todos)
💰 Budget: $2.34 / $10.00 used this session
🚀 Launching claude...

claude › I can see you're building a billing API for the
monetization module. Based on your roadmap...
```

Magic Moments

The init wizard scans your README, git history, and existing docs to pre-fill your roadmap and memory with real context — not generic placeholders.

V0.9.2 · TEAM + CONSENSUS

Run agents in parallel. Reach consensus.

Dispatch fires an agent as a background job — captured to a log file, tracked by the job store. Multiple agents run simultaneously while your terminal stays free. New in v0.9.2: panel decisions with `synlynk decide`.

synlynk dispatch

```
$ synlynk dispatch claude --task "Write unit tests for the billing module"
✓ Dispatched job-a3f2  (claude)  running in background

$ synlynk dispatch agy --task "Review billing API for edge cases"
✓ Dispatched job-b7e1  (agy)  running in background

# Both agents working simultaneously — terminal is yours
```

Monitor all jobs

synlynk jobs

```
$ synlynk jobs

• job-a3f2  claude  running 2m14s
• job-b7e1  agy  running 1m58s
```

Tail a job's output

synlynk logs

```
$ synlynk logs job-a3f2

Writing test_billing.py...
✓ test_charge_success
✓ test_refund_partial
→ 12 / 12 tests passing
```

Multi-Agent Consensus v0.9.2

Ask multiple agents the same question and get a structured consensus. Panel results are collected, compared, and optionally written as signed Decision records.

synlynk decide

```
$ synlynk decide "SQLite or Markdown for decisions?" --panel claude,codex,agy --record

Dispatching panel: claude, codex, agy…

claude → SQLite — queryable, joins with stories
codex  → Markdown — human-readable, diffable in PRs
agy    → SQLite — aligns with existing state.db pattern

✓ Consensus: SQLite (2/3) — Decision record written
```

- **Smart Routing — Live since v0.5.0**
  Dispatch automatically routes to the best-performing agent for each task domain, based on historical capability scores in `state.db`. Pass `--story <id>` to track work items and build the capability ledger.

SENTINEL & BUDGET GUARD

Your safety net, always on.

Synlynk watches every agent invocation for danger signals and surfaces alerts before they become expensive mistakes.

Budget Guard

Set spending limits in `.synlynk/config.json`. Synlynk blocks the next exec the moment you'd exceed your limit. Cost shown after every run.

.synlynk/config.json

```
{
  "limit_usd": 10.00,
  "limit_requests": 100
}
```

budget pulse

```
── Budget Pulse ──────────────
Cost:     $2.34 / $10.00
Requests: 14 / 100
Tokens:   2,840 in · 480 out
```

Sentinel Patterns

Sentinel watches your telemetry log and fires alerts automatically for five danger patterns.

FLATLINE **Same command failed 3× in a row**
Possible hallucination loop. Intervene before more tokens burn.

LOOP **Same command succeeded 5× in <10 min**
Likely an automated loop running unproductively. Investigate.

QUOTA **Quota exhaustion detected in output**
Output matched a known quota-exhausted phrase. Switch agent.

ZOMBIE **Job PID dead or stalled >30 min**
Reconciled automatically on the next run.

DRIFT **Instruction file edited outside synlynk**
SHA mismatch on tracked files. Use `instructions diff/update`.

Managing Alerts

sentinel commands

```
$ synlynk sentinel list

[CRITICAL]  FLATLINE      `synlynk exec claude` failed 3x  — 2026-06-14 09:41
[WARN]    SUCCESS_LOOP  Same command 5x in 4.2 min     — 2026-06-14 10:02

$ synlynk sentinel clear --code FLATLINE
✓ Cleared FLATLINE alerts
```

ALL COMMANDS

Everything at a glance.

Setup & Context

COMMAND	WHAT IT DOES
`synlynk init`	Bootstrap project-docs/, all agent files, and budget config. Accepts `--docs-dir`.
`synlynk exec <agent>`	Run agent with scoped context, ## Relevant Files, and ## How to Verify injected.
`synlynk checkpoint`	Regenerate `.synlynk/context.md` from current project-docs.
`synlynk scan`	Build source-map.md from codebase symbols. `--deep` for full re-scan.
`synlynk status`	Dashboard: costs, active alerts, running jobs.
`synlynk upgrade`	Check for and apply CLI updates.

Team v0.9.2

COMMAND	WHAT IT DOES
`synlynk join`	Onboard as a new team member: seed devlog from git history, regenerate AI context files, print team digest.
`synlynk team status`	Show all members, story assignments, last-active, and token budget consumption.
`synlynk decide <topic>`	Multi-agent consensus panel. `--panel` selects agents. `--record` writes signed Decision.

Sentinel

COMMAND	WHAT IT DOES
`synlynk sentinel list`	Show all active alerts with severity, code, and timestamp.
`synlynk sentinel clear`	Clear alerts. Target with `--severity` or `--code`.

Stories & Capability v0.5.0

COMMAND	WHAT IT DOES
`synlynk story create`	Create story in state.db. `--domain`, `--tokens`, `--priority`.
`synlynk story list`	List stories with status, domain, assignee, and capability score.
`synlynk score`	Capability score breakdown per agent × domain.
`synlynk score attest`	Manually attest a quality score with a signed reason.

Dispatch & Jobs v0.4.0

COMMAND	WHAT IT DOES
`synlynk dispatch <agent>`	Fire background job with smart routing. Accepts `--task` and `--story`.
`synlynk jobs`	List jobs (running / completed / failed). `--all` includes history.
`synlynk logs <job-id>`	Tail job output. Use `--tail N` to limit lines.
`synlynk run --trio`	Architect → Builder → Verifier pipeline with context hand-off.

Instructions & Agent v0.4.1

COMMAND	WHAT IT DOES
`synlynk instructions status`	Show status of all tracked files (ok / missing / drifted).
`synlynk instructions update`	Regenerate synlynk section and reset SHA in manifest.
`synlynk agent run`	Run support engineer agent: collect signals, file GH issues, draft fix PRs.
`synlynk agent --install-cron`	Register launchd/systemd timer for scheduled agent runs.

project-docs/	roadmap.md · memory.md · todo.md · costs.md · devlogs/<user>.md · decisions/
.synlynk/	config.json · context.md · instructions.json · telemetry.json · source-map.md · logs/
state.db	~/.synlynk/projects/<git-root-hash>/state.db (stories · capability_ratings · source_symbols · jobs)

WHAT'S COMING NEXT

The road to v1.0.

Synlynk evolves from an agent wrapper into a full OS for multi-agent development. Each release adds a new layer of the stack.

v0.7 **Static Scan Quality** ✓ Live · June 2026
Language-agnostic source scanner · ## Source Architecture in every exec session · synlynk scan

v0.8 **Support Engineer Agent** ✓ Live · June 2026
5 signal collectors · GH issue filing · draft fix PRs · .agents/ config · cron install

v0.9 **Kernel Fixes + Team** ✓ You are here · June 2026
Scoped context · Ed25519 signing · synlynk join · team status · decide consensus

v0.9.3 **Async Daemon** Planned · Aug 2026
synlynk daemon · launchd/systemd · job queue · HTTP context server localhost:27471

v1.0 **Community Layer + Public Launch** Planned · Sep 2026
Workgroup relay · signed capability ledger · pipx/Homebrew · synlynk.com